
Binary Reverse Engineering Lecture Material

Course: *Introduction to Vulnerability Research*

Author name: *Dr. Bernard Lampe*

Due date: *October 31th, 2019*

Introduction

1. Why reverse?
2. When would you reverse (give example situations)?
3. How does reversing fit into the larger VR picture?
4. What are the legal aspects of reversing?

Overview of How Compilers Work

1. Go from C to binary and back. Give them an idea of what happens under the hood.
2. Front end of compiler, (Lexical analysis (lex), grammar(yacc), semantic analysis(type checking))
3. Back end of compiler, (optimization, register spilling, code generation)

Tools

1. Reversing tools like Radare, IDA, Binja, objdump, etc

Architectures x86, x86_64, ARM, ARM64, MIPS, PPC

1. Things about arch's.

Disassemblers

1. Possibly show how disassemblers are written.

Decompilers

1. Possibly show how decompilations are not unique.
2. Show pitfalls of decompilers, like function inlining and optimizations

Reversing Examples

1. Example x86, x86_64, ARM, ARM64, MIPS and PPC. Maybe compiler a non-trivial (but smaller) program to show how it goes to assembly and the differences in each.
2. I'd like real world examples here.
3. Perhaps tear down an embedded system and then show the following reversing.
4. Show example of an application process doing network stuff.
5. Show example of reversing the bootloader to setup a chip.
6. Other ideas, reverse a proprietary filesystem, proprietary algorithm?

Listing 1: Code Example Listing

```
1 int main()  
2 {  
3     printf("hello_worldz\n");  
4     return 0;  
5 }
```

Conclusion

Conclusions here....

References

1. references here....